Sri Sivasubramaniya Nadar College of Engineering,
Kalavakkam - 603 110
(An Autonomous Institution, Affiliated to Anna University, Chennai)

Course Code	UIT2702	Course Name	Cloud and Distributed Computing
Regulation	2021	Academic Year	**2026-2027**
Semester		VII	
Department Offering the Course		Information Technology	
Exercise: 03		Message passing and distributed mutual exclusion	

Name: Rahul Akshath Rajendran

Reg No: 3122235002095

Sem / Sec: VII B

Exp. No: 03 **Date: 22-07-2026**

MESSAGE PASSING AND DISTRIBUTED MUTUAL EXCLUSION

AIM

To implement message passing between distributed processes using sockets, and to achieve distributed mutual exclusion among those processes using the Ricart-Agrawala algorithm, the Token Ring algorithm and the Centralized Coordinator algorithm.

QUESTION

Task 1 - Message Passing System:

- Create a server that listens for connections and allows multiple clients to connect via sockets.

- Each client can send and receive messages, enabling communication between distributed processes.

Task 2 - Distributed Mutual Exclusion:

Implement the Ricart-Agrawala algorithm, Token Ring Algorithm and Centralized Coordinator Algorithm for mutual exclusion among distributed processes. For each algorithm:

- Processes request access to a critical section (CS) by sending messages.
- Ensure that only one process enters the CS at a time.
- Use message passing to request, grant, and release access to the CS.

ALGORITHM / STEPS

Task 1 - Message Passing System

1. Server: create a TCP socket, set the SO_REUSEADDR option, bind it to ("localhost", 6000) and listen for up to five clients.
2. Server: accept connections in an endless loop. Add every accepted socket to a list of clients and start a separate thread for that client, so that many clients can be served at the same time.
3. Server (client thread): read the first message, which is the name of the process, and broadcast a join notification to all the other clients.
4. Server (client thread): keep receiving messages from that client, print them on the server console and broadcast them to every other connected client.
5. Server (client thread): when the received message is empty the client has disconnected, so remove its socket from the list, close it and broadcast a leave notification.
6. Client: create a TCP socket, connect to ("localhost", 6000) and send the user name as the first message.
7. Client: start a background thread that continuously receives messages from the server and prints them, so that receiving does not block sending.
8. Client: in the main thread read the messages typed by the user and send them to the server until the user types "exit", then close the socket.

Task 2 (a) - Ricart-Agrawala Algorithm (fully distributed, permission based)

1. Every process Pi maintains a Lamport logical clock, a listening socket on the port 7000 + i, a counter of replies received and a list of deferred replies.

2. When Pi wants to enter the critical section it increments its clock, saves that value as the timestamp of its request, sets requesting = True and sends the message REQUEST(timestamp, i) to all the other N-1 processes.

3. When Pi receives REQUEST(tj, j) it updates its clock as clock = max(clock, tj) + 1 and then takes a decision: if Pi is currently inside the critical section, or Pi is itself requesting with a smaller (timestamp, id) pair, then the reply is deferred and j is added to the deferred list; otherwise a REPLY is sent to Pj immediately.

4. Ties between two requests with the same timestamp are broken using the process id, so the ordering of requests is total.

5. When Pi receives a REPLY it increments its reply counter.

6. As soon as the counter reaches N-1, that is every other process has given permission, Pi enters the critical section.

7. When Pi leaves the critical section it clears the in_cs flag and sends a REPLY to every process present in its deferred list, which allows the next process in timestamp order to proceed. Because a process defers a reply whenever it has an older request, only one process can collect all N-1 replies at a time, which guarantees mutual exclusion.

Task 2 (b) - Token Ring Algorithm (token based)

1. The N processes are arranged in a logical ring. Process i listens on the port 8000 + i and its successor in the ring is the process (i mod N) + 1.

2. A single token is created by process 1 when it starts. The token is the permission to enter the critical section.

3. A process that needs the critical section sets its want_cs flag (in the program this is done by pressing the Enter key) and waits for the token to arrive.

4. When a process receives the token it checks its want_cs flag. If the flag is set it enters the critical section, executes it, leaves it and clears the flag. If the flag is not set it simply does nothing.

5. In both cases the process then forwards the token to its successor in the ring, so the token keeps circulating.

6. Since exactly one token exists in the ring and a process may enter the critical section only while it is holding the token, mutual exclusion is guaranteed. Every process gets the token once per round, so starvation cannot occur.

Task 2 (c) - Centralized Coordinator Algorithm

1. One process is elected as the coordinator. It listens on the port 9000 and maintains a busy flag, a FIFO queue of waiting processes and the socket of every process.
2. Every ordinary process connects to the coordinator when it starts and keeps the connection open.
3. When a process needs the critical section it sends the message REQUEST <id> to the coordinator and blocks until it gets an answer.
4. On receiving a REQUEST the coordinator checks the busy flag. If the critical section is free it sets busy = True and sends GRANT to that process; otherwise it appends the process id to the queue and sends no reply, which makes the process wait.
5. A process enters the critical section only after it receives GRANT, executes it, and then sends RELEASE <id> to the coordinator.
6. On receiving a RELEASE the coordinator removes the first process from the queue and sends GRANT to it; if the queue is empty it simply sets busy = False.
7. Because the coordinator keeps only one outstanding GRANT at any instant, only one process can be inside the critical section, and the FIFO queue ensures that every waiting request is eventually served.

CODE

Task 1 - Message Passing System

server.py

```
import socket

import threading
```

```
clients = []

def broadcast(message, sender=None):
    for client in clients[:]:
        if client != sender:
            try:
                client.send(message.encode())
            except:
                clients.remove(client)

def handle_client(conn, addr):
    name = conn.recv(1024).decode()
    print(name, "joined")

    broadcast(name + " joined the chat", conn)

    while True:
        try:
            msg = conn.recv(1024).decode()
            if not msg:
                break

            print(name + ":", msg)
```

```
            broadcast(name + ": " + msg, conn)

        except:
            break

    if conn in clients:
        clients.remove(conn)

    conn.close()
    print(name, "left")
    broadcast(name + " left the chat")


server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("localhost", 6000))
server.listen()
server.settimeout(1)      # Check every 1 second

print("Server started... Press Ctrl+C to stop.")
try:
    while True:
        try:
            conn, addr = server.accept()
            clients.append(conn)
```

```
            threading.Thread(target=handle_client, args=(conn, addr),
daemon=True).start()
        except socket.timeout:
            continue


except KeyboardInterrupt:
    print("\nStopping server...")


finally:
    for client in clients:
        client.close()
    server.close()
     print("Server stopped.")
```

client.py

```
import socket
import threading

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(("localhost", 6000))

name = input("Enter your name: ")
client.send(name.encode())

def receive():
    while True:
        try:
            print(client.recv(1024).decode())
        except:
```

```
            break

threading.Thread(target=receive, daemon=True).start()

print("Type messages (type 'exit' to quit)")

while True:
    msg = input()

    if msg == "exit":
        break

    client.send(msg.encode())

client.close()
```

Execution: run "python3 server.py" in one terminal and "python3 client.py" in two or more other terminals. Type a name in each client and then exchange messages.

Task 2 (a) - Ricart-Agrawala Algorithm

ricart_agrawala.py

```
import socket
import threading
import time
import sys

ME = int(sys.argv[1])
N = 3
PORT = 7000 + ME

replies = 0
requesting = False
deferred = []
```

```
def send(pid, msg):
    try:
        s = socket.socket()
        s.connect(("localhost", 7000 + pid))
        s.send(msg.encode())
        s.close()
    except:
        pass

def handle(msg):
    global replies

    parts = msg.split()

    if parts[0] == "REQUEST":
        pid = int(parts[1])

        print("REQUEST from P%d" % pid)

        if requesting:
            deferred.append(pid)
        else:
            send(pid, "REPLY %d" % ME)

    elif parts[0] == "REPLY":
        replies += 1
        print("REPLY from P%d (%d/%d)" % (int(parts[1]), replies, N - 1))

def listener():
    server = socket.socket()
    server.bind(("localhost", PORT))
    server.listen(5)
    server.settimeout(1)
```

```
    try:
        while True:
            try:
                conn, addr = server.accept()
                msg = conn.recv(1024).decode()
                conn.close()
                handle(msg)
            except socket.timeout:
                pass
    except KeyboardInterrupt:
        pass
    finally:
        server.close()

threading.Thread(target=listener, daemon=True).start()

print("Process", ME, "started")

try:
    while True:

        input("Press Enter to request Critical Section...")

        requesting = True
        replies = 0

        for i in range(1, N + 1):
            if i != ME:
                send(i, "REQUEST %d" % ME)

        while replies < N - 1:
            time.sleep(0.1)
```

```
        print(">>> P%d ENTERED Critical Section" % ME)
        time.sleep(5)
        print("<<< P%d LEFT Critical Section" % ME)

        requesting = False

        for pid in deferred:
            send(pid, "REPLY %d" % ME)

        deferred.clear()

except KeyboardInterrupt:
    print("\nProcess stopped.")
```

Task 2 (b) - Token Ring Algorithm

token_ring.py

```
import socket
import threading
import sys
import time

ME = int(sys.argv[1])
N = 3

NEXT = ME % N + 1
PORT = 8000 + ME

want = False
```

```
def send_token():
    try:
        s = socket.socket()
        s.connect(("localhost", 8000 + NEXT))
        s.send(b"TOKEN")
        s.close()
    except:
        pass

def use_token():
    global want

    print("P%d received TOKEN" % ME)

    if want:
        print("P%d ENTERED Critical Section" % ME)
        time.sleep(3)
        print("P%d LEFT Critical Section" % ME)
        want = False

    print("P%d passed TOKEN to P%d" % (ME, NEXT))
    time.sleep(1)
    send_token()
```

```
def request_cs():
    global want

    while True:
        try:
            input("Press Enter to request Critical Section...")
            want = True
        except KeyboardInterrupt:
            break

threading.Thread(target=request_cs, daemon=True).start()

server = socket.socket()
server.bind(("localhost", PORT))
server.listen(5)
server.settimeout(1)

print("Process", ME, "started")

# Process 1 starts with the token
if ME == 1:
    time.sleep(2)
    use_token()
```

```
try:
    while True:
        try:
            conn, addr = server.accept()
            msg = conn.recv(1024).decode()
            conn.close()

            if msg == "TOKEN":
                use_token()

        except socket.timeout:
            pass

except KeyboardInterrupt:
    print("\nProcess stopped.")

finally:
    server.close()
```

Task 2 (c) - Centralized Coordinator Algorithm

coordinator.py

```
import socket
import threading
```

```
clients = {}
queue = []
busy = False

def grant(pid):
    clients[pid].send(b"GRANT")
    print("GRANT -> P%d" % pid)

def handle(conn):
    global busy

    while True:
        try:
            msg = conn.recv(1024).decode()
            if not msg:
                break

            cmd, pid = msg.split()
            pid = int(pid)

            clients[pid] = conn

            if cmd == "REQUEST":
```

```
            print("REQUEST from P%d" % pid)

            if busy:
                queue.append(pid)
            else:
                busy = True
                grant(pid)

        elif cmd == "RELEASE":
            print("RELEASE from P%d" % pid)

            if queue:
                grant(queue.pop(0))
            else:
                busy = False

    except:
        break

conn.close()

server = socket.socket()
server.bind(("localhost", 9000))
server.listen(5)
```

```
server.settimeout(1)

print("Coordinator started...")

try:
    while True:
        try:
            conn, addr = server.accept()
            print("Process connected:", addr)
            threading.Thread(target=handle, args=(conn,), daemon=True).start()
        except socket.timeout:
            pass

except KeyboardInterrupt:
    print("\nCoordinator stopped.")

finally:
    for c in clients.values():
        c.close()
    server.close()
```

process.py

```
import socket
import sys
import time
```

```
ME = int(sys.argv[1])

client = socket.socket()
client.connect(("localhost", 9000))

print("Process", ME, "connected")

try:
   while True:

      input("Press Enter to request Critical Section...")

      client.send(f"REQUEST {ME}".encode())
      print("REQUEST sent")

      reply = client.recv(1024).decode()

      if reply == "GRANT":
         print(">>> P%d ENTERED Critical Section" % ME)

         time.sleep(5)

         print("<<< P%d LEFT Critical Section" % ME)

         client.send(f"RELEASE {ME}".encode())

except KeyboardInterrupt:
   print("\nProcess stopped.")

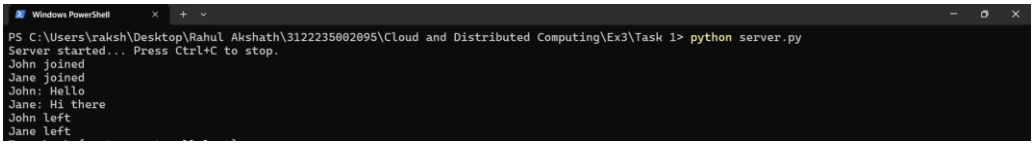
finally:
   client.close()
```

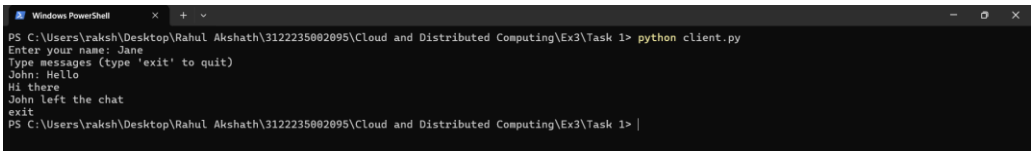
Steps to execute Task 2

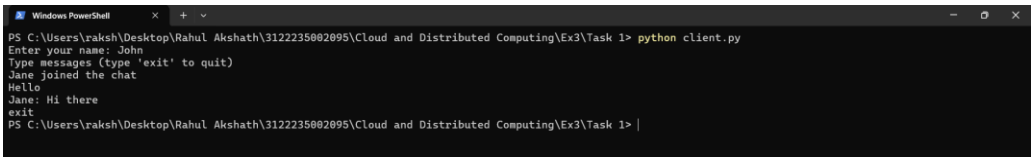
- Ricart-Agrawala: open three terminals and run "python3 ricart_agrawala.py 1", "python3 ricart_agrawala.py 2" and "python3 ricart_agrawala.py 3". Press Enter in two terminals at almost the same time; one process enters the critical section while the other one waits for the deferred reply.
- Token Ring: open three terminals and start "python3 token_ring.py 2" and "python3 token_ring.py 3" first, then "python3 token_ring.py 1", which creates the token. Press Enter in any terminal to request the critical section and watch the token circulate.
- Centralized Coordinator: run "python3 coordinator.py" first, then run "python3 process.py 1", "python3 process.py 2" and "python3 process.py 3" in three other terminals. Press Enter in more than one process to see the requests being queued and granted one after another.

OUTPUT SCREENSHOTS

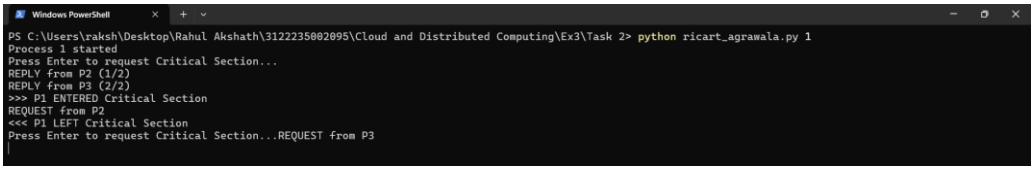
Task 1 - Message Passing System

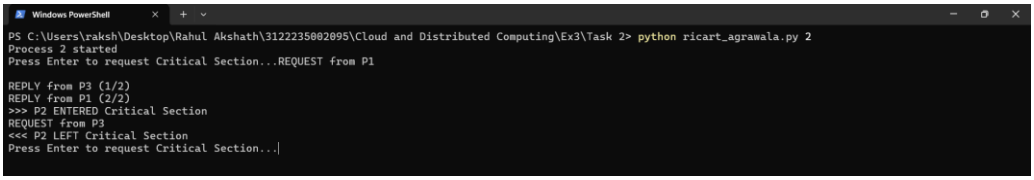


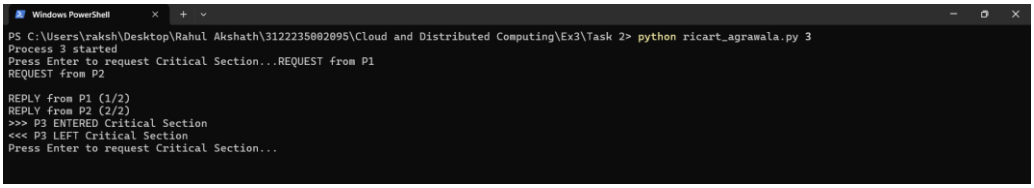




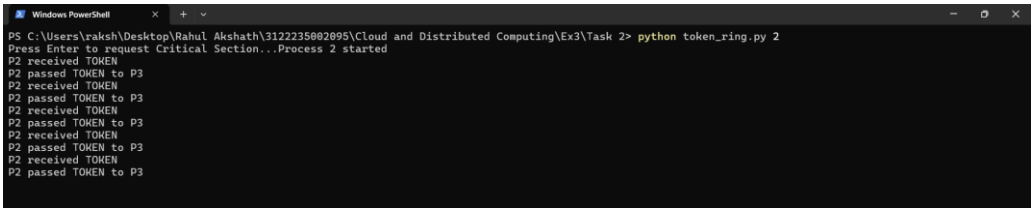
Task 2 (a) - Ricart-Agrawala Algorithm

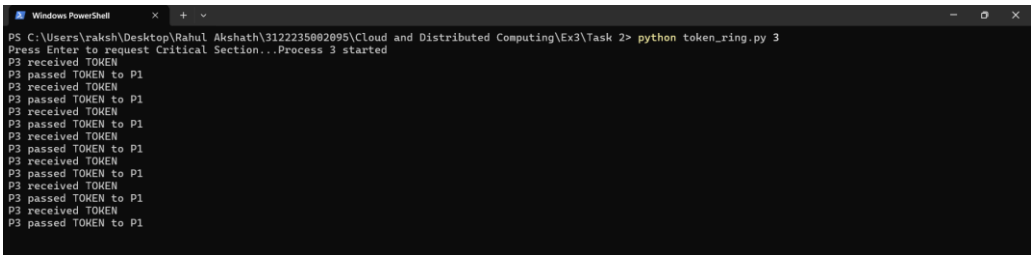


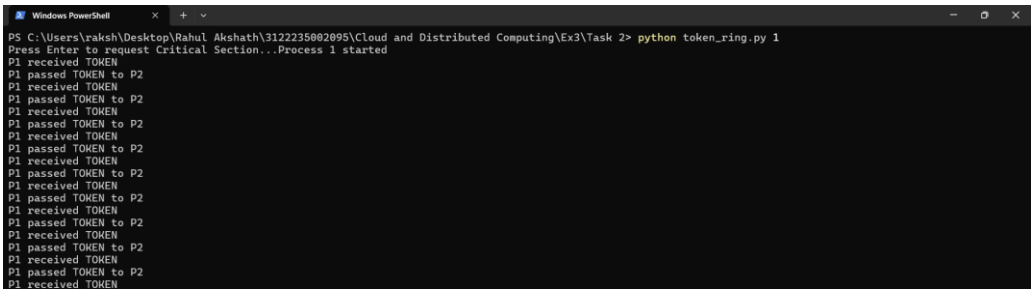




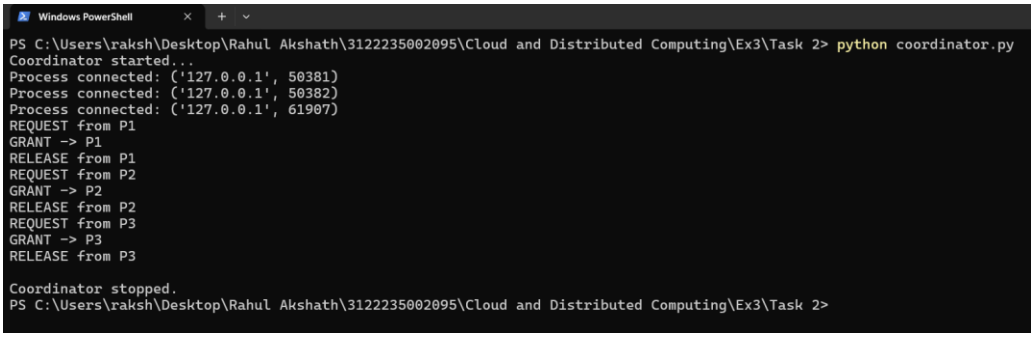
Task 2 (b) - Token Ring Algorithm

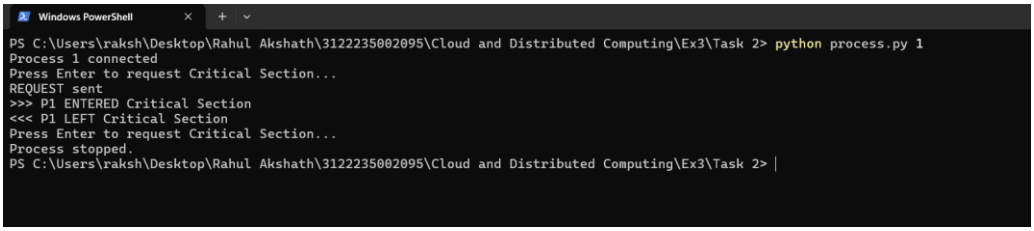


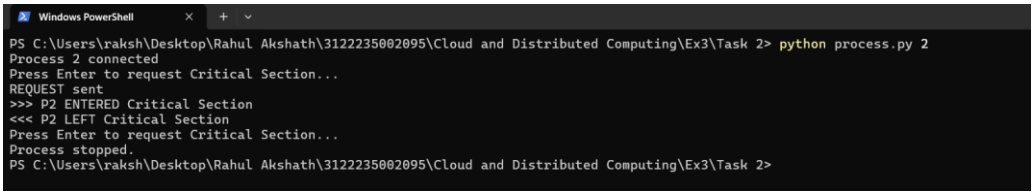


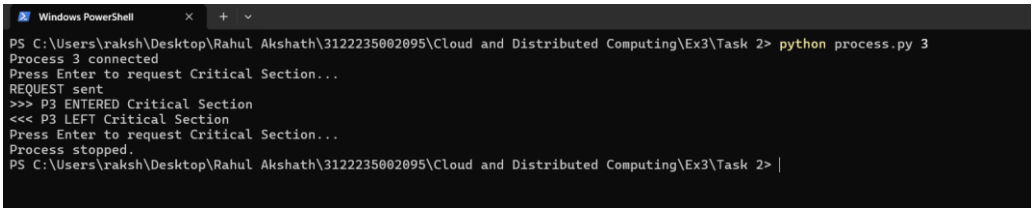


Task 2 (c) - Centralized Coordinator Algorithm









CONCLUSION

A message passing system and three distributed mutual exclusion algorithms were implemented successfully using TCP sockets. In Task 1 a multithreaded server allowed several client processes to connect at the same time and broadcast messages to one another, which shows how distributed processes communicate purely by exchanging messages without any shared memory. In Task 2 the same message passing mechanism was used to coordinate access to a critical section. The Ricart-Agrawala algorithm is fully distributed and uses 2(N-1) messages per entry with timestamp ordering, the Token Ring algorithm uses a single circulating token and therefore needs no request messages at all but adds waiting time for the token, and the Centralized Coordinator algorithm is the simplest with only three messages per entry but the coordinator is a single point

of failure and a possible bottleneck. In every case the output confirmed that only one process was inside the critical section at any instant, and that every waiting request was eventually granted, so mutual exclusion and freedom from starvation were achieved.